МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №3
по курсу “Компьютерная графика”

Текстурирование

Выполнил(а): Василянская Алёна Николаевна
Группа: М8О-314БВ-24
Преподаватель: В. Д. Бахарев

Москва, 2026

Условие

Цель работы – научиться загружать изображения из файловой системы, создавать из них текстуры и накладывать их на объекты с помощью текстурных координат вершин и сэмплера. Работа продолжает лабораторную работу 2 и выполнена на C++ с Vulkan, GLFW и ImGui; программа работает в реальном времени, а текстура каждой модели выбирается в интерфейсе из выпадающего списка.

Вариант 5 – сфера.

По базовому заданию на сферу и пол из второй работы накладывается текстура, которая служит альбедо вместо сплошного цвета и цветов вершин. Вершины, кроме позиции и нормали, несут текстурные координаты, построенные процедурно. Картинки загружаются из файлов библиотекой stb_image. Свойства моделей хранятся в одном динамическом uniform-буфере, а сдвиг до свойств нужной модели задаётся при привязке набора дескрипторов. Вершинные и индексные буферы находятся в видеопамяти (VRAM) и заполняются через промежуточный (staging) буфер командами копирования GPU.

Кроме того, реализованы дополнительные задания 1 и 2:

1) загружаются все картинки из папки textures, для сферы и для пола текстура выбирается из выпадающего списка; у каждой текстуры свой набор дескрипторов;

2) для сферы реализовано нетривиальное сэмплирование – «воронка»: текстурные координаты закручиваются вокруг центра картинки, одно и то же изображение читается пять раз по разным координатам и результаты усредняются, а цвет внутри воронки модулируется; эффект зависит от времени и включается флагом в push-константах.

Возможности второй работы сохранены: камера, все типы источников и их показ, перенос, поворот, масштаб и анимация сферы, пол. Флажок Vertex colors и цвет альбедо убраны – основной цвет теперь берётся из текстуры.

Метод решения

Текстурные координаты. Для сферы координаты получаются из тех же индексов, что и позиция: для вершины в i-м ряду по широте и j-м по долготе

$$u = 1 - j/N, \quad v = i/N,$$

где $N = 9$ – число сегментов, так что сетка по-прежнему содержит 100 вершин. Координата v растёт от северного полюса к южному, поэтому верх картинки оказывается сверху. Угол θ растёт от оси X к оси Z, то есть при взгляде снаружи справа налево, поэтому u отсчитывается в обратную сторону – иначе картинка была бы зеркальной. Столбец вершин на шве в сетке уже повторён: у него u = 0 и u = 1, поэтому последняя полоса треугольников не сжимает в себя всю текстуру. У пола углы квадрата получают координаты от (0, 0) до (1, 1), причём v = 0 на дальнем от камеры крае. В вершинном шейдере координаты умножаются на число повторов текстуры модели (Tiling), а сэмплер работает в режиме адресации REPEAT, поэтому вне отрезка [0, 1] текстура повторяется. Число повторов целое, чтобы картинка совпадала на шве сферы; по умолчанию у сферы 2 × 1, так как долгота охватывает 360°, а широта – 180°.

Загрузка изображений. При запуске загружаются все файлы png, jpg, jpeg, bmp, tga и gif из папки textures в порядке имён; stb_image декодирует каждую картинку в массив пикселей RGBA8. Для каждой через VMA создаётся VkImage в памяти GPU (двумерное, один MIP-уровень, оптимальное расположение, использование TRANSFER_DST и SAMPLED) и VkImageView. Формат – VK_FORMAT_R8G8B8A8_SRGB: пиксели в файлах закодированы в sRGB, при выборке GPU переводит их в линейные значения, в которых считается освещение, а swapchain формата sRGB кодирует итоговый цвет обратно. Для всех текстур используется один VkSampler с билинейной фильтрацией, адресацией REPEAT и без MIP-уровней.

Видеопамять и staging-буфер. Память VRAM процессору недоступна, поэтому данные проходят через staging-буфер в видимой с CPU памяти (64 МБ, использование TRANSFER_SRC). Он заполняется как линейный аллокатор с выравниванием блоков по 16 байт, после записи память сбрасывается функцией vmaFlushAllocation. Для загрузки созданы отдельный пул командных буферов, командный буфер и fence. Для изображения записываются барьер, переводящий его из layout UNDEFINED

в TRANSFER_DST_OPTIMAL, копирование vkCmdCopyBufferToImage и второй барьер – в SHADER_READ_ONLY_OPTIMAL, после которого изображение читается во фрагментном шейдере. Для буфера записываются vkCmdCopyBuffer и барьер памяти, после которого данные доступны на стадии чтения вершин. Командный буфер отправляется в очередь, и CPU ждёт fence. Вершинные и индексные буферы сферы, пола и визуализации источников созданы в памяти GPU без доступа с CPU. Текстуры, сфера и пол загружаются при запуске, а вершины визуализации источников – каждый кадр после ожидания fence кадра.

Динамический uniform-буфер. Свойства моделей – матрица модели, цвет и коэффициент блика и число повторов текстуры, всего 96 байт в раскладке std140 – лежат в одном uniform-буфере массивом: сфера – элемент 0, пол – элемент 1. Сдвиг динамического буфера должен быть кратен minUniformBufferOffsetAlignment из VkPhysicalDeviceLimits, поэтому шаг массива равен размеру структуры, округлённому вверх до этого выравнивания a: stride = ⌈96/a⌉·a. Дескриптор буфера – динамический (UNIFORM_BUFFER_DYNAMIC) с диапазоном в одну структуру, а при отрисовке модели в vkCmdBindDescriptorSets передаётся динамический сдвиг index·stride.

Наборы дескрипторов. Набор 0 (сцена) не изменился по сравнению со второй работой. Набор 1 (материал) содержит динамический uniform-буфер свойств моделей (binding 0) и пару «изображение + сэмплер» (binding 1). Таких наборов создаётся столько, сколько загружено текстур, и все они ссылаются на один uniform-буфер. Пул дескрипторов рассчитан на 1 + N наборов: один uniform- и два storage-дескриптора, N динамических uniform-дескрипторов и N пар «изображение + сэмплер». Модель рисуется с набором выбранной для неё текстуры и своим сдвигом, поэтому одна текстура может быть одновременно и у сферы, и у пола.

Воронка. Параметры эффекта передаются push-константами (16 байт, фрагментный шейдер): флаг включения, время t, сила закрутки s и радиус R. Для пола все они нулевые, и выборка обычная. Для сферы в каждом повторе текстуры берётся точка p относительно его центра (0,5; 0,5), r = |p|. Внутри круга радиуса R точка поворачивается на угол

$$\varphi = s\cdot(1 - r/R)^2\cdot\sin t,$$

наибольший у центра и спадающий до нуля к краю круга; множитель sin t меняет направление закрутки со временем – это нетривиальное

сэмплирование. Затем текстура читается пять раз с углами φ, $0{,}9\varphi$, $0{,}8\varphi$, $0{,}7\varphi$ и $0{,}6\varphi$, и результаты усредняются – изображение смазывается вдоль закрутки. Внутри круга цвет умножается на оттенок $0{,}7 + 0{,}3\cdot\cos(t + 2\pi r + \delta)$, где $\delta = 0$, $2\pi/3$ и $4\pi/3$ для красного, зелёного и синего каналов, взятый с весом $1 - r/R$: в центре оттенок полный, а к краю круга плавно сходит на нет – по воронке бегут цветные кольца (модуляция цвета). Вне круга $\varphi = 0$ и множитель равен 1, поэтому на границах повторов и на шве сферы картинка непрерывна. Полученный цвет служит альбедо $K_d = K_a$ в модели Блинна–Фонга из лабораторной работы 2.

Интерфейс. В окне Sphere в разделе Material появились выпадающий список Texture и повторы Tiling, а флажок Vertex colors и цвет Albedo убраны; новый раздел Swirl содержит флажок Enable swirl и параметры Twist (сила закрутки), Swirl radius и Swirl speed. В окне Scene раздел Floor получил те же Texture и Tiling.

Реализация на Vulkan. Конвейер моделей (VkPipelineLayout с наборами 0 и 1 и диапазоном push-констант) рисует треугольники с отсечением задних граней и тестом глубины; вершина состоит из позиции и нормали (по три float) и текстурных координат (два float). Набор сцены привязывается один раз, а перед каждым vkCmdDrawIndexed (сфера, пол) – набор текстуры модели с её динамическим сдвигом и push-константы. Конвейеры визуализации источников не изменились. При завершении работы уничтожаются сэмплер, изображения с их VkImageView, буферы, пул и раскладки дескрипторов, а также пул командных буферов, fence и staging-буфер загрузки.

Результаты

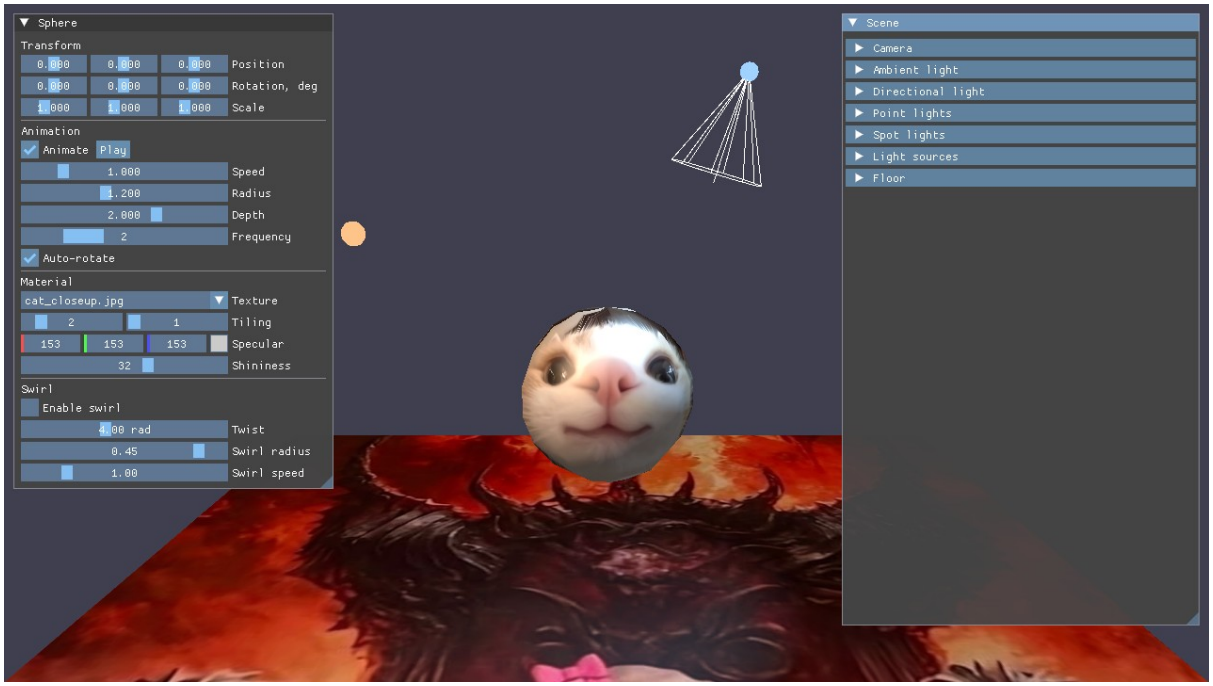


Рисунок 1 – Текстуры на сфере и полу

Сфере назначена картинка cat_closeup.jpg с повторами 2 × 1, полу – cat_throne.jpg без повторов; анимация остановлена кнопкой Pause. По долготе картинка повторяется дважды, по широте – один раз, и морда кота обращена к камере. Освещение то же, что во второй работе: оранжевый точечный источник и голубой прожектор подкрашивают текстуру.

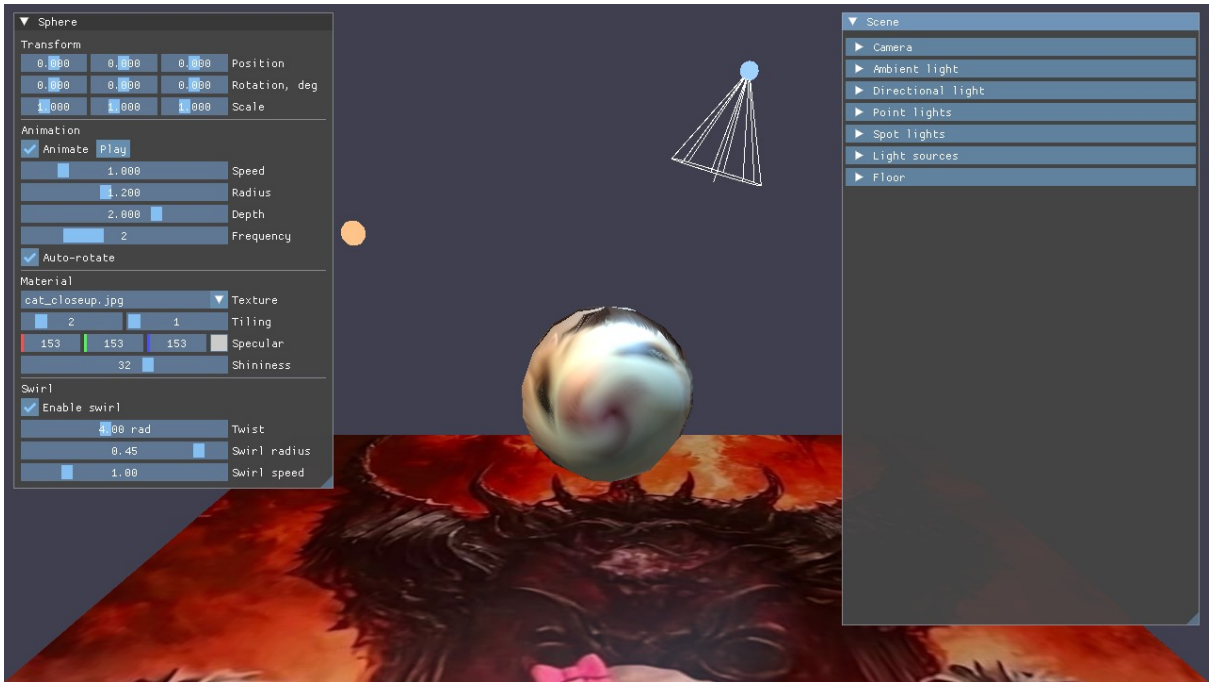


Рисунок 2 – Воронка на сфере

Включён эффект Swirl: сила закрутки 4 рад, радиус 0,45, скорость 1. Морда кота закручена вокруг центра повтора, контуры смазаны вдоль закрутки, а по воронке идут цветные кольца; со временем закрутка меняет направление.

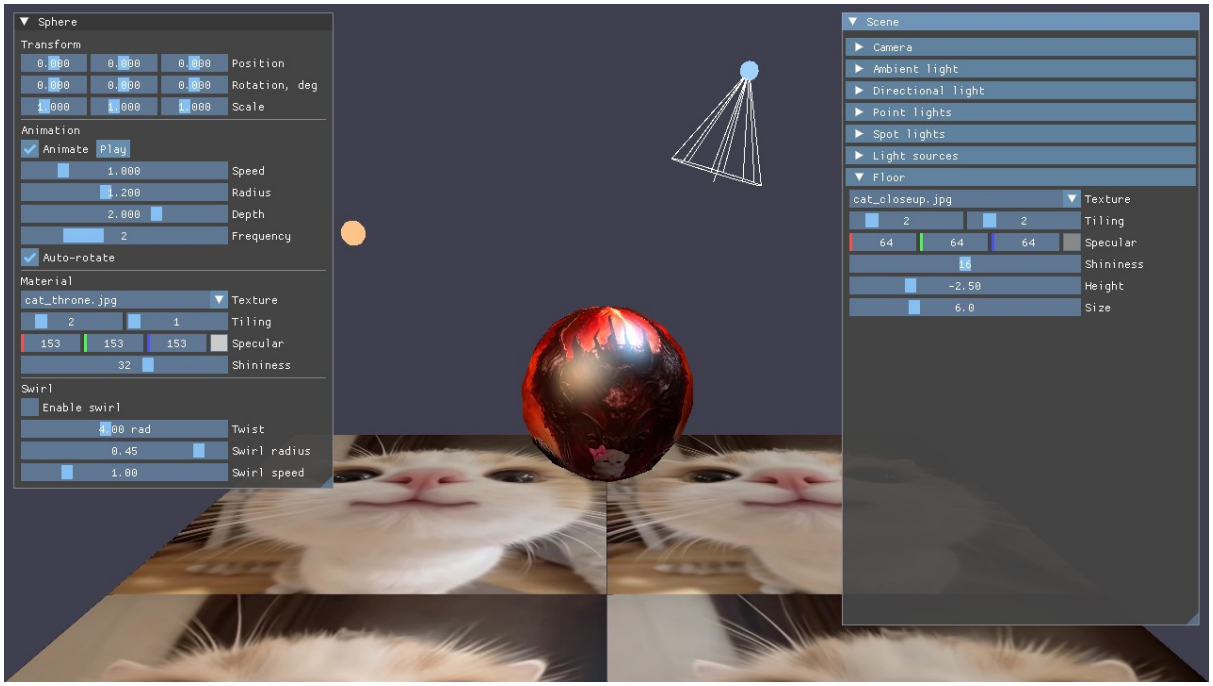


Рисунок 3 – Другие текстуры у сферы и пола

Сфере выбрана картинка cat_throne.jpg, полу – cat_closeup.jpg с повторами 2 × 2. Каждая модель рисуется с набором дескрипторов своей текстуры; в окне Scene открыт раздел Floor.

Выводы

В работе освоено текстурирование в Vulkan: изображения загружаются из файлов, копируются в видеопамять через staging-буфер одноразовым командным буфером с ожиданием fence, переводятся барьерами между layout и читаются во фрагментном шейдере через сэмплер. Построены процедурные текстурные координаты сферы с учётом шва и направления обхода, рассмотрены билинейная фильтрация, режим адресации REPEAT и хранение цветных текстур в формате sRGB. Вершинные и индексные буферы перенесены в видеопамять, отдельные uniform-буферы моделей заменены одним динамическим со сдвигом, выровненным по minUniformBufferOffsetAlignment, а для каждой текстуры создан свой набор дескрипторов. С помощью push-констант реализован зависящий от времени эффект «воронка» с несколькими выборками одной текстуры и модуляцией цвета.